TECHNICAL PROPOSAL / v1.0

Club Tech Projects

Three flagship initiatives • 9-week sprint • deadline 30 June

01 **SMRITI**

An offline AI notebook — your second memory, on your phone.

02 **SETU-RAKSHA**

A unified, real-time digital identity risk-propagation engine for India.

03 **AVAHAN**

A modular humanoid robot built in four engineered phases.

Each project is designed to be useful, socially meaningful, technically substantial, and shippable by a student team within a defined semester scope.

Executive Summary

This document proposes three flagship projects for the club's upcoming cycle. The projects are deliberately diverse in scope — a consumer mobile application, a national-scale systems research challenge, and a multi-phase humanoid robotics roadmap — but share a common thread: each addresses a problem that matters, demands real engineering depth, and leaves the club with a concrete, demoable artifact at the end.

The projects are sized so that a team of 4–8 student engineers can ship a defensible version-one in a single semester, with clear milestones for extension into research papers, patents, grants, or product launches.

At a glance

#	Project	Domain	Team Size	Timeline
01	Smriti	Mobile App / On-device AI	5–6	9 weeks (deadline 30 Jun)
02	Setu-Raksha	Cybersecurity / Graph ML	5–7	9 weeks (deadline 30 Jun)
03	Avahan	Robotics / Embedded / AI	6–10	9 weeks (Phase 1 + start of Phase 2)

Why these three?

- **Range of skills:** mobile, ML, embedded, robotics, networking, graph algorithms, full-stack — nobody on the team is left without a meaningful track to learn.
- **Range of difficulty:** Smriti is the most contained and ship-ready. Setu-Raksha is research-grade with publication potential. Avahan is the broadest project and will continue beyond the deadline as a flagship build.
- **Real-world relevance:** each project addresses a documented gap in the Indian context — privacy-preserving AI, financial fraud, and human-robot interaction in non-Western settings.
- **Defensible outcomes:** every project produces something the club can show — a working app on the Play Store, a paper-ready prototype, or a humanoid that greets visitors at the techfest.

PROJECT 01

Smriti

An offline AI notebook — your second memory, on your phone.

Concept

Smriti is a mobile application that turns any collection of personal documents — PDFs, lecture notes, images, audio recordings, textbooks — into a queryable, conversational knowledge base. The user drops files in; the app processes them locally; from then on, the user can ask natural-language questions or request summaries, and the app answers grounded in the user's own material. **Everything runs on the device. No internet required after setup. No data ever leaves the phone.**

> **The one-line pitch**
>
> Drop your textbooks, notes, and recordings into Smriti — then ask any question, anytime, anywhere, with no internet and complete privacy.

Why it matters

- **Privacy by design.** Sensitive material — medical reports, legal documents, personal journals, exam notes — never touches a server. This is a hard differentiator from every cloud-based AI tool today.
- **Works without internet.** Useful in classrooms with poor connectivity, in remote areas, on flights, and during exam prep when distraction-free study matters.
- **Universal user base.** Every student, researcher, professional, and elderly user has documents they need to query. The product needs no convincing.
- **Demonstrates a frontier capability.** On-device LLM inference is genuinely new in 2025–26; building on it positions the club ahead of the curve.

How it works — the pipeline

1. Ingest	User drops files (PDF, DOCX, TXT, images, audio). The app routes each to the right extractor.
2. Extract	PDFBox / pdf.js for documents, ML Kit OCR for images, whisper.cpp for audio — all on-device.
3. Chunk	Extracted text is split into ~500-token semantic chunks with metadata (source, page, position).

4. Embed	Each chunk is converted to a vector using a small quantized embedding model (~25 MB).
5. Index	Vectors are stored in a local vector database (ObjectBox or sqlite-vss).
6. Retrieve	When the user asks a question, the question is embedded and the most relevant chunks are pulled.
7. Answer	Retrieved chunks + the question are passed to a local LLM (Gemma 2B / Phi-3 / Llama 3.2 1B) which answers, with citations back to the source page.

Technical stack

App framework	Flutter (cross-platform) or native Kotlin (Android-first for performance).
LLM runtime	MediaPipe LLM Inference API or llama.cpp Android bindings.
Language model	Gemma 2B Q4 / Phi-3 Mini / Llama 3.2 1B — quantized to ~1–2 GB on disk.
Embedding model	all-MiniLM-L6-v2 quantized — ~25 MB, fast on CPU.
Vector store	ObjectBox (built-in vector search) or SQLite + sqlite-vss.
Document parsing	Apache PDFBox (Android) for PDFs, docx4j for Word documents.
OCR	Google ML Kit — free, on-device, supports Indian scripts.
Audio transcription	whisper.cpp (tiny or base model) running locally.

Realistic performance on a mid-range phone

Targeted at devices with 6–8 GB RAM and a Snapdragon 7-series or equivalent SoC:

Operation	Expected performance
First-time embedding of a 50-page PDF	~30 seconds (one-time, then cached)
RAG retrieval per query	Under 1 second
LLM token generation (Gemma 2B Q4)	5–10 tokens / second
End-to-end answer to a question	10–30 seconds
Cold start (loading model into RAM)	3–5 seconds

Module split — recommended team of 5–6

Module	Owner role	Key responsibilities
1. App shell + UI	Mobile dev	Notebook UI, file picker, chat interface, settings, model download manager.
2. File ingestion	Backend / parsing	PDF, DOCX, image, audio extractors. Robust error handling.
3. Embeddings + vector DB	ML / data	Chunking strategy, embedding model integration, vector index management.
4. LLM integration	ML / systems	Model loading, inference, token streaming, memory management.
5. RAG orchestration	Full-stack	Query → retrieve → prompt → answer pipeline. Citation back to source.
6. QA, polish, deploy	Generalist / lead	Beta testing, Play Store packaging, documentation, demo.

9-week timeline (April 29 → June 30)

Week	Milestone
1	App shell, file picker UI, PDF text extraction working end-to-end.
2	Chunking + embedding pipeline; vector DB integrated; retrieval verified manually.

Week	Milestone
3	LLM integrated via MediaPipe; basic Q&A; flow operational on a single document.
4	Multi-document support, citations back to source pages, summarization mode.
5	OCR for images (ML Kit) and audio transcription via whisper.cpp.
6	UI polish, model download manager, settings, cold-start optimization.
7	Closed beta with 10–15 college students; performance profiling on different phones.
8	Bug fixes, fallback to smaller model on low-RAM devices, documentation.
9	Play Store submission, demo recording, club showcase (by 30 June).

Risks and mitigations

- **Model size vs. Play Store limits.** APK size capped at 200 MB. Mitigation: ship the app empty, download the model on first launch over Wi-Fi.
- **Thermal throttling on phones.** Long inference makes phones hot. Mitigation: stream tokens, batch reasonably, show a clear 'thinking' state.
- **Low-end device support.** Phones with under 4 GB RAM cannot load Gemma 2B. Mitigation: offer Llama 3.2 1B Q4 (~700 MB) as a fallback model.
- **iOS complexity.** Apple's policies on shipping LLMs are stricter. Mitigation: launch Android-first, treat iOS as a follow-up.
- **Trust in answers.** Users won't trust ungrounded LLM output. Mitigation: every answer must show the source chunks/pages used, clickable.

Deliverables

- A published Android app on the Play Store.
- A 5-minute demo video showcasing offline operation.
- Open-source repository with full documentation.
- Internal report covering on-device LLM benchmarks across phones.

PROJECT 02

Setu-Raksha

A unified, real-time digital identity risk-propagation engine for India.

The problem

India operates one of the world's largest digital public infrastructures. Aadhaar, UPI, the banking system, telecom SIM authentication, and the entire fintech layer are interconnected through a deep digital identity graph. The systems are **linked**; the fraud-detection engines protecting them are **not**.

> **The core insight**
>
> Fraud isn't a single-system problem — it propagates across systems. But every detection engine today operates within the boundary of a single institution. There is no real-time, cross-network risk-propagation model for India's digital identity ecosystem.

How a real attack chains across systems

Consider a typical attack:

- **Step 1.** Fraudster executes a SIM swap against the target's mobile number.
- **Step 2.** They gain access to OTPs sent to that number.
- **Step 3.** They reset the target's UPI PIN using OTP-based authentication.
- **Step 4.** They transfer funds through a network of mule accounts.
- **Step 5.** The funds are layered through small transactions across multiple accounts to launder.

At no single point in this chain does any one system see enough signal to stop the attack. The telecom operator sees a SIM swap. The bank sees an OTP-validated transaction. The UPI app sees a PIN reset. Each is internally normal. Cross-system, the chain is screaming.

Structural vulnerabilities Setu-Raksha targets

Identity centralization	One Aadhaar-linked mobile number is the single point of compromise across banking, UPI, government schemes, and telecom. A breach at one node cascades.
Mule account networks	Fraud rarely happens in isolation — it flows through dormant accounts, low-KYC accounts, and temporary SIM registrations. These form graph structures invisible to rule-based detection.

Synthetic identities	Fraudsters combine a real Aadhaar with a fake SIM, alternate device fingerprint, and engineered transaction patterns to create identities that look statistically clean.
Reactive detection	Banks detect fraud after the transaction. Telecom detects after a SIM swap complaint. UPI apps detect after anomaly. There is no predictive cross-network risk score.

The proposed solution

Setu-Raksha is a research prototype of a **cross-network risk-propagation engine**. It models the Indian digital identity ecosystem as a heterogeneous graph — nodes are identities, devices, SIMs, accounts, and transactions; edges are the relationships between them — and applies graph machine learning to detect risk patterns that no single institution could see on its own.

System architecture (research prototype)

Synthetic data layer	A simulated digital identity graph mimicking realistic Indian patterns — Aadhaar links, SIM-bank-UPI chains, mule networks. Built from public reports and adversarial scenarios. (Real data is, correctly, inaccessible — synthetic is the right approach for a research prototype.)
Graph store	Neo4j or TigerGraph for the identity-transaction graph. PostgreSQL for tabular metadata.
Graph ML engine	Graph Neural Networks (GraphSAGE / GAT) to learn embeddings of identities and detect anomalous subgraphs. PyTorch Geometric or DGL.
Risk-propagation algorithm	When any node is flagged (SIM swap, dormant-account activation, unusual login), risk scores propagate across the graph in real time, raising alerts on connected nodes.
Streaming pipeline	Kafka or Redpanda to simulate real-time events from multiple sources (telecom, bank, UPI). Apache Flink for stateful stream processing.
Visualization layer	Next.js + a graph viz library (Cytoscape.js or vis-network) to show, live, how risk propagates through the network when an attack chain triggers.
Adversarial testbed	A scripted attack simulator that runs SIM-swap-to-mule-laundering chains, so the team can measure detection lead time.

Why this is a national-scale problem worth working on

- Billions of UPI transactions are processed every month — small percentage failures translate to massive absolute fraud volumes.
- Fintech onboarding is accelerating; KYC quality varies wildly across providers.
- Digital literacy is rising faster than cybersecurity awareness, especially in rural expansion.
- There is no academic prototype, public benchmark, or open-source tool for cross-network identity risk in the Indian context. The club would be filling a real gap.

This is not a software bug. It is a system-design asymmetry problem *— and the right response is a system-level architectural research contribution.*

Technical stack

Graph database	Neo4j (community edition) or TigerGraph.

Graph ML	PyTorch Geometric, DGL, GraphSAGE / GAT models.
Streaming	Kafka or Redpanda + Apache Flink for stream processing.
Backend / API	FastAPI in Python, async-first.
Frontend	Next.js + Cytoscape.js or vis-network for live graph visualization.
Synthetic data	Custom generator using the Faker library + realistic Indian-context patterns.
Containerization	Docker Compose for the whole stack so it can be demoed end-to-end on a laptop.

Module split — recommended team of 5–7

Module	Focus
1. Synthetic data + simulator	Generate realistic identity graphs and attack scenarios.
2. Graph store + ingestion	Load identities, devices, SIMs, accounts into Neo4j; ingest streaming events.
3. Graph ML modeling	Train GNN models to embed nodes and detect anomalies.
4. Risk-propagation engine	Real-time score propagation when a node is flagged.
5. Streaming pipeline	Kafka + Flink: handle high-throughput simulated events.
6. Visualization + dashboard	Live graph view of an attack chain unfolding and being detected.
7. Evaluation + paper	Benchmark detection lead-time, false-positive rates; write the research report.

9-week timeline (April 29 → June 30)

Week	Milestone
1	Define synthetic data schema. Stand up Neo4j locally. Begin identity-graph generator (Aadhaar–SIM–bank–UPI patterns).
2	Generator complete; full synthetic graph loaded. Build attack-scenario simulator (SIM swap, mule chain, layering).

Week	Milestone
3	Train baseline GNN (GraphSAGE) for node embeddings on the synthetic graph; verify embeddings cluster sensibly.
4	Anomaly-detection model on top of GNN embeddings; first benchmark numbers on detecting injected attacks.
5	Risk-propagation algorithm — scores cascade across the graph in real time when a node is flagged.
6	Streaming pipeline — Kafka producers simulate live events from telecom, bank, and UPI sources.
7	Visualization dashboard in Next.js — live graph view of an attack chain unfolding and being detected.
8	Benchmark suite — measure detection lead time, false-positive rate, propagation latency across scenarios.
9	Final demo, write the technical report, prepare paper draft (by 30 June).

Deliverables

- A working prototype that demonstrates cross-network risk propagation on a simulated Indian digital identity graph.
- A live visualization showing how an attack chain (SIM swap → UPI compromise → mule laundering) propagates and is detected.
- A benchmark dataset of synthetic attack scenarios.
- A research-grade technical report suitable for submission to a venue such as IC3IA, COMSNETS, or a security workshop.
- A clear policy ask — what regulatory or technical bridge between systems would make this defense deployable in production.

PROJECT 03

Avahan

A modular humanoid robot built in four engineered phases.

Approach

Building a humanoid robot end-to-end is a multi-year endeavour. The right approach for a club is not to attempt everything at once — it is to **structure development across capability layers**, with each layer producing a clean, demoable milestone. We propose four phases:

Phase	Layer	Deliverable
1	Perception + basic interaction	Robot recognizes club members and greets them by name.
2	Conversational intelligence	Robot answers club-related questions through structured dialogue.
3	Mobility + autonomous navigation	Robot moves autonomously within the club area.
4	Social intelligence	Robot follows people, recognizes emotion, hosts events.

Phase 1 — Identity recognition & smart greeting

Objective. The robot detects known club members from a camera feed and initiates a personalized audio interaction.

Core capabilities

- Real-time face detection from an HD camera feed.
- Face recognition against a club member database.
- Voice greeting via on-board speakers.
- Unknown-person handling (a default polite greeting and prompt to register).
- Local logging of attendance events.
- Confidence threshold to avoid misidentification, with multi-face detection and basic sentiment cues.

Stack

Hardware	HD camera module, microphone array, speaker system, edge compute (Raspberry Pi 5 or NVIDIA Jetson Nano/Orin).
Software	OpenCV, face_recognition / dlib, lightweight CNN if needed, pyttsx3 / Coqui TTS, SQLite member database.

Outcome

By the end of Phase 1, the robot identifies club members on sight, greets them by name, and logs attendance. This is a clean, projector-ready demo milestone.

Phase 2 — Intelligent communication

Objective. The robot becomes an AI-powered information assistant for the club. Rather than a general chatbot, it is structured as a **knowledge-based conversational agent** bounded to the club's domain — events, schedules, member roles, ongoing projects, directions, and FAQs.

Capabilities

- Answer questions about club events, workshop schedules, member roles, and ongoing projects.
- Provide directions inside the building.
- Handle a curated FAQ database with confidence-scored intent classification.
- Voice-based interaction (speech in, speech out).
- Short-term context memory across a single conversation.
- Admin panel to update FAQs and content without redeploying.

Optional advanced layer

A retrieval-based response system grounded in the club's documents and a small **local LLM (offline inference)** running on the Jetson — this gives the robot the ability to handle out-of-FAQ questions intelligently without a cloud dependency. The infrastructure overlaps directly with Project 1 (Smriti), so component reuse is possible.

Stack

Hardware	Microphone array, speaker, NVIDIA Jetson (recommended for LLM inference).
Software	Vosk or Whisper for offline speech-to-text, an intent classifier (fastText / DistilBERT), MongoDB or SQLite for FAQ data, a dialogue manager, Coqui TTS for output.
Optional	llama.cpp running a 3B–7B quantized model with RAG over the club's document corpus.

Outcome

By the end of Phase 2, the robot holds structured conversations, answers club-related questions reliably, and can be updated by non-engineers via an admin panel. It is now a real interactive assistant.

Phase 3 — Mobility & autonomous navigation

Objective. The robot moves autonomously within a defined indoor environment.

Core capabilities

- Obstacle detection.
- Path planning (A* on an occupancy grid).
- Indoor SLAM-based navigation.
- Balance control if a bipedal form factor is pursued.

- Target-based movement (e.g., 'follow me' or 'go to the entrance').

Strategic recommendation

> **Choose the right form factor early**
>
> Full bipedal humanoid walking is a multi-year research problem. We strongly recommend **Option A — wheeled mobile base + humanoid upper torso** for the first iteration. This keeps the AI, perception, and conversational layers usable while sidestepping gait control. Bipedal locomotion (Option B) can be pursued as a parallel research track.

Stack

Hardware	Motor drivers, wheel motors or servo joints, IMU, ultrasonic sensors, optional LiDAR, depth camera (Intel RealSense / OAK-D).
Software	ROS 2, SLAM module (RTAB-Map or Cartographer), A* or RRT* path planning, PID motor control, sensor-fusion stack.

Outcome

By the end of Phase 3, the robot navigates the club area autonomously, avoiding obstacles, and can interact dynamically with people in the space.

Phase 4 — Autonomous social intelligence

Objective. Layer advanced social capabilities on top of the navigation and conversational stack.

- **Follow-person mode.** The robot tracks and follows a designated user through the space.
- **Emotion detection.** Reading facial expressions and adapting tone of response.
- **Event-hosting mode.** The robot anchors club events — introducing speakers, transitioning agenda items.
- **Demo presentation mode.** A scripted, timed walkthrough for visitors and judges.
- **Gesture recognition.** Responding to wave, point, and stop gestures naturally.

Roadmap summary

Phase	Focus	What the club gains
1	Perception (face recognition + greeting)	A robot that recognizes and greets members.
2	Intelligence (conversational AI)	A robot that answers questions and represents the club.
3	Mobility (navigation + movement)	A robot that moves through the space on its own.
4	Social intelligence	A robot that hosts events, follows people, and reads the room.

Why phasing matters

Each phase produces a self-contained demoable system. If the project loses momentum mid-Phase-3, the club still has a fully functional Phase-2 robot that recognizes members and converses with them. This is the single biggest reason ambitious robotics projects fail — they go for the moonshot first, and end up with nothing demoable. Phasing protects against that.

Deliverables across phases

- **Phase 1.** Working face-recognition greeter, member database, attendance log.
- **Phase 2.** Voice-driven Q&A; with admin panel, optional offline LLM for open-ended questions.
- **Phase 3.** Autonomous indoor navigation with obstacle avoidance and follow mode.
- **Phase 4.** Emotion-aware social robot capable of hosting club events and demos.

9-week timeline (April 29 → June 30)

The deadline allows Phase 1 to be fully shipped with a head-start on Phase 2. Phases 3 and 4 continue as a flagship multi-semester track beyond June.

Week	Milestone
1	Hardware procurement (Jetson / Pi 5, HD camera, mic array, speaker, mounting). OS + dev environment setup.
2	Camera pipeline live; real-time face detection working (OpenCV / MediaPipe). Frame rate profiled.
3	Member database schema; face-recognition model trained on 15–20 club members; recognition accuracy >95%.
4	Confidence threshold tuning, multi-face detection, unknown-person fallback. Attendance logging to SQLite.
5	TTS engine integrated; personalized greeting pipeline complete. **Phase 1 demo-ready.**
6	Phase 2 starts — offline speech-to-text (Vosk / Whisper) integrated; mic-array audio capture stable.
7	Intent classifier + FAQ database (events, members, projects). Dialogue manager with short-term context.
8	Admin panel for non-engineers to update FAQs. End-to-end voice Q&A; loop working reliably.
9	Polish, internal demo, public showcase (by 30 June). Phases 3–4 plan handed off.

Comparative summary

Dimension	Smriti	Setu-Raksha	Avahan
Domain	Mobile / on-device AI	Cybersecurity / graph ML	Robotics / embedded / AI
Time to first demo	~3 weeks	~5 weeks	~5 weeks (Phase 1)
Final deadline	30 June (9 weeks)	30 June (9 weeks)	30 June (9 weeks, Phase 1+)
Team size	5–6	5–7	6–10
Hardware needed	Phones (already owned)	Laptops / 1 server	Camera, edge AI device, motors, base
Public outcome	Play Store launch	Research paper / open-source	Live robot at fests
Funding angle	Product / startup	Research grant	Innovation / IP grant

Recommendation for the club

All three projects are worth pursuing — they appeal to different skill profiles and cover different kinds of public outcomes (a shipped product, a research contribution, a flagship demo). A pragmatic execution plan runs them in parallel as three sub-teams under a shared 30 June deadline, with infrastructure reused where it makes sense — the on-device LLM stack from Smriti can be plugged directly into Avahan's Phase 2 conversational layer.

> **Suggested first move**
>
> Kick off all three tracks the same week. Smriti and Avahan Phase 1 are scoped to produce demoable artifacts within the first 3–5 weeks, building the team's confidence and the club's momentum early. Setu-Raksha runs as a parallel research track with the same 30 June endpoint, ending in a working prototype + paper draft.

Next steps

- Identify project leads and team members for each track this week.
- Lock hardware budget immediately — Avahan (edge device, camera, motors, base) needs procurement done by Week 1.
- Set up shared GitHub organization with one repository per project.

- Schedule weekly cross-project syncs to share components and avoid duplicated work.
- Fixed checkpoint dates: **Week 3** first internal demo, **Week 6** mid-point review, **Week 9 (30 June)** public showcase.
